

DISPENSA DELLA LEZIONE

useEffect in React

Effetti dopo il render, dipendenze, cleanup con return, timer e richieste
HTTP con fetch e async/await.

CAPITOLO 1

Che cos'è useEffect

NOTA

useEffect è un Hook di React che serve per eseguire del codice dopo il rendering del componente.

Si usa quando il componente deve fare qualcosa oltre a mostrare il JSX, per esempio scrivere in console, avviare un timer o caricare dati.

Quando un componente React viene renderizzato, il suo compito principale è costruire l'interfaccia. A volte, però, serve anche eseguire un'azione collegata al componente ma separata dal JSX. Questo tipo di azione viene chiamato effetto.

useEffect permette di scrivere questi effetti in un punto preciso del componente. In questo modo si separa meglio la parte che mostra contenuti dalla parte che esegue operazioni aggiuntive.

La forma più semplice di **useEffect** usa una funzione e un array vuoto. In questo caso l'effetto parte una sola volta, subito dopo il primo render del componente.

esempio

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    console.log("Componente caricato");
  }, []);

  return <h1>Homepage</h1>;
}
```

CAPITOLO 2

L'array delle dipendenze

NOTA

Il secondo argomento di `useEffect` è l'array delle dipendenze. Serve per dire a React quando rieseguire l'effetto.

Se una dipendenza cambia, React esegue di nuovo l'effetto dopo il nuovo render.

L'array delle dipendenze controlla il comportamento dell'effetto. Se l'array è vuoto, l'effetto parte solo all'inizio. Se dentro l'array c'è una variabile, l'effetto parte all'inizio e poi ogni volta che quella variabile cambia.

Questo è utile quando un effetto dipende da uno state o da una prop. Per esempio, se un componente deve reagire al cambiamento di un contatore, si inserisce quel contatore nell'array delle dipendenze.

Capire bene questa parte è importante, perché `useEffect` non va usato in modo casuale. L'idea è collegare l'effetto ai dati che lo fanno cambiare.

esempio

```
import { useEffect, useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Il valore di count è cambiato:", count);
  }, [count]);

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Aumenta</button>
    </div>
  );
}
```

CAPITOLO 3

Il return dentro useEffect

NOTA

Il **return** dentro **useEffect** serve per restituire una funzione di pulizia, chiamata **cleanup**.

Questa funzione viene eseguita prima che il componente si smonti oppure prima che l'effetto venga eseguito di nuovo.

Questo passaggio è fondamentale quando l'effetto avvia qualcosa che deve essere fermato in seguito. Per esempio, un timer, un listener o una richiesta HTTP possono continuare a lavorare anche quando il componente non c'è più.

Se non si pulisce il tutto, si rischiano comportamenti sbagliati, aggiornamenti inutili o errori. Il **return** di **useEffect** non restituisce quindi JSX: restituisce una funzione che React userà per fare ordine.

La funzione di **cleanup** è molto utile anche con le richieste asincrone. In quel caso si può fermare la richiesta con **AbortController** oppure evitare di aggiornare lo state quando il componente non è più attivo.

esempio

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    const id = setInterval(() => {
      console.log("Timer attivo");
    }, 1000);

    return () => {
      clearInterval(id);
    };
  }, []);

  return <p>Timer avviato</p>;
}
```

CAPITOLO 4

useEffect con timer

NOTA

`useEffect` si usa spesso con `setInterval()` o `setTimeout()` per avviare timer collegati al componente.

Quando il timer non serve più, è importante fermarlo con una funzione di pulizia nel `return`.

Un timer continua a lavorare anche dopo essere stato avviato. Se il componente sparisce oppure l'effetto viene eseguito di nuovo, il timer precedente può restare attivo e creare comportamenti sbagliati.

Per questo `useEffect` può restituire una funzione finale. Questa funzione viene usata per ripulire ciò che era stato avviato dall'effetto, per esempio un intervallo.

Questo meccanismo è molto importante non solo per i timer, ma anche per altri casi in cui il componente apre una connessione o registra un listener.

esempio

```
import { useEffect, useState } from "react";

function App() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const intervalId = setInterval(() => {
      setSeconds((currentValue) => currentValue + 1);
    }, 1000);

    return () => {
      clearInterval(intervalId);
    };
  }, []);

  return <p>Secondi: {seconds}</p>;
}
```

CAPITOLO 5

useEffect e richieste HTTP con fetch

NOTA

Quando un componente deve recuperare dati da un server, spesso si usa `fetch()` dentro `useEffect`.

Durante l'attesa della risposta si può usare uno state di caricamento per mostrare un loader o un messaggio temporaneo.

Una richiesta HTTP serve per comunicare con un server e ricevere dati. In React questa operazione viene spesso eseguita come effetto, perché deve partire dopo il render iniziale del componente.

Il metodo `fetch()` invia la richiesta e restituisce una Promise. Quando la risposta arriva, di solito viene trasformata in JSON con `response.json()` e poi salvata nello state.

In questa situazione è utile aggiungere anche uno state come `loading`. Questo valore permette di distinguere il momento in cui i dati non sono ancora arrivati dal momento in cui il contenuto è pronto per essere mostrato.

esempio

```
import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  return (
    <div>
      {loading ? (
        <p>Caricamento in corso...</p>
      ) : (
        <ul>
          {users.map((user) => (
            <li key={user.id}>{user.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```